ChemCS: Integrating Constraint Satisfaction and Diffusion Models for Locally Valid Molecular Generation

Sreepranav Chetlapalli

Abstract

Molecular generation with diffusion models has shown promise for drug discovery and materials design, yet these models frequently produce chemically invalid structures such as atoms with too many bonds, reactions that violate mass conservation, or mismatched formal charges. Most existing approaches address this by filtering invalid outputs after generation is complete, which wastes computation and provides no mechanism for mid-trajectory correction.

This paper presents ChemCS, a proof-of-concept system that integrates a constraint-satisfaction supervisor directly into the reverse-diffusion loop of a graph neural network (GNN) molecular generator. At every denoising step the supervisor decodes the current molecular state, verifies chemical axioms (valency limits, mass conservation, charge balance, and atom counts), and either commits the step, applies targeted repairs, or backtracks to a previously valid state. The constraint checker uses the Z3 SMT solver from Microsoft Research to formulate and verify chemical rules as satisfiability queries.

The generator itself is a lightweight two-layer GNN that operates on atom-level feature vectors and an adjacency matrix of bond orders. A companion PyTorch training pipeline adds differentiable constraint penalties for valency and element conservation to the standard denoising loss, with a curriculum schedule that ramps these penalties gradually to avoid destabilizing early training.

In a small-scale exploratory benchmark of three reactions (SN2, combustion, acid-base) with 50 random seeds each, *every* run uses a freshly initialized, **untrained** GNN (random weights). Under that setup the supervisor enforces 100% valency validity (every atom respects its bond-count limit) compared with 12–22% for unsupervised generation. Full conservation (mass, charge, atom counts) stays at 0% for both modes here because random weights still predict wrong element types. Separately, we run the PyTorch trainer to obtain a training-loss curve on a few small molecules; those optimized weights are **not** loaded into this benchmark, so the table isolates the supervisor from the effect of learning.

1 Introduction

Generating novel molecular structures is a central challenge in computational chemistry. Applications range from *de novo* drug design, where candidate molecules must satisfy pharmacological constraints, to materials science, where target properties such as band gap or thermal stability guide the search. In recent years, diffusion models, originally developed for image generation, have been adapted to molecular graphs with encouraging results.

A diffusion model generates a sample by starting from pure noise and iteratively refining it through a sequence of learned denoising steps. When applied to molecules the noise is added to atom features and bond orders, and the reverse process gradually recovers a structured molecular graph. The difficulty is that the neural network performing these updates has no built-in knowledge of chemistry: it can propose carbon atoms with five bonds, reactions where mass appears or disappears, or products whose total charge does not match the reactants.

The standard mitigation is post-hoc filtering: generate many candidate molecules, discard the invalid ones, and keep what remains. This strategy has two significant drawbacks. First, it is wasteful. If the model has a low validity rate, the vast majority of computation is thrown away. Second, and more importantly, it misses the opportunity to guide the generative process. If an error is detected at step 40 of a 50-step trajectory, there is no reason to wait until step 50 to discover and discard it.

ChemCS addresses this gap by wrapping the diffusion model in a supervisor loop that enforces chemical axioms at every reverse step. This work is a proof-of-concept: the experiments use a small GNN and a limited set of reactions, and the results demonstrate the viability of the approach rather than state-of-the-art molecular generation. The key contributions are:

1. A constraint-satisfaction module that encodes four fundamental chemical rules (valency limits, mass conservation, charge balance, and atom-count conservation) as queries to the Z3 satisfiability modulo theories (SMT) solver.
2. A supervisor loop that intercepts each denoising step, runs constraint checks, and responds with one of three actions: *commit* (accept the step), *correct* (apply targeted repairs), or *backtrack* (revert to the last valid state).
3. A constraint-aware training objective that supplements the standard denoising loss with differentiable penalties for valency violations and element-count mismatches, introduced through a curriculum schedule.
4. A test suite and benchmark framework for reproducible evaluation.

Related Work

Several recent systems apply diffusion models to molecular graphs. Hoogeboom et al. introduced an E(3) equivariant diffusion model that generates molecules in 3D coordinate space, enforcing rotational symmetry but not chemical bonding rules. Vignac et al. proposed DiGress, a discrete diffusion model that operates directly on categorical node and edge attributes and scales to datasets with over one million molecules. Both approaches generate molecules without mid-trajectory constraint enforcement and rely on post-hoc validity filtering. ChemCS is complementary to these methods: it contributes the supervisor architecture (constraint checking, correction, backtracking) rather than the generative model itself, and could in principle wrap any graph-based diffusion backbone.

2 Background

This section provides a brief overview of the two main technical foundations that ChemCS builds upon: denoising diffusion models for graph-structured data and satisfiability modulo theories for constraint checking.

2.1 Denoising Diffusion Models on Graphs

A denoising diffusion probabilistic model (DDPM) defines a forward process that progressively corrupts a data sample x_0 by adding Gaussian noise over T timesteps, producing a sequence $x_0, x_1, \ldots, x_T$ where x_T is approximately standard Gaussian noise. Generation works by learning the reverse process: a neural network ε_θ is trained to predict the noise added at each step, so that starting from $x_T \sim \mathcal{N}(0, I)$ the model can iteratively recover a clean sample x_0.

The amount of noise at each timestep is controlled by a variance schedule $\{\beta_t\}_{t=1}^{T}$. A key quantity is the cumulative signal-retention factor $\bar{\alpha}_t = \prod_{s=1}^{t}(1 - \beta_s)$, which determines how much

of the original signal remains at step t. When $\bar{\alpha}_T \approx 0$ the data is fully destroyed; when $\bar{\alpha}_1 \approx 1$ almost no noise has been added.

For molecular graphs, the data x_0 consists of two components: a matrix of atom features (element type, bond count, charge) and an adjacency matrix of bond orders. The forward process adds continuous Gaussian noise to atom features and randomly corrupts bond orders. The reverse process uses a graph neural network to predict the clean state from the noisy one. ChemCS supports both linear and cosine noise schedules.

2.2 Satisfiability Modulo Theories (SMT) and Z3

Satisfiability modulo theories (SMT) extends Boolean satisfiability (SAT) with reasoning over richer domains such as integers, real numbers, and arrays. An SMT solver takes a set of logical constraints and determines whether any variable assignment satisfies all of them simultaneously.

Z3 is a high-performance SMT solver developed by Microsoft Research. In ChemCS, Z3 is used to encode chemical rules as constraints: for example, the statement "for every atom i, the total number of bonds must not exceed the allowed valency for that element and charge" becomes an integer-arithmetic constraint that Z3 can check or refute in microseconds.

For the four rules currently implemented, a simple procedural checker would suffice. The choice of Z3 is a practical one: it provides a uniform API for expressing constraints of varying complexity, and it reduces the risk of bugs in hand-written checking logic. It also leaves the door open for more complex constraints (such as ring-size limits or aromaticity rules) that would benefit from a real solver. The current implementation does not exercise this extensibility, so Z3 should be understood as a convenient tool choice rather than a strict architectural requirement.

3 Problem Formulation

3.1 Molecular Representation

A molecule is represented as a set of atoms $\{a_1, a_2, \ldots, a_n\}$, where each atom a_i carries four attributes:

- Element e_i: the chemical element (e.g., C, N, O).
- Bond count b_i: the sum of bond orders to neighbouring atoms.
- Formal charge q_i: the integer formal charge (e.g., -1 for bromide).
- Implicit hydrogen count h_i: hydrogens not explicitly represented in the graph.

Bonds between atoms are stored in a symmetric adjacency matrix $A \in \{0, 1, 2, 3\}^{n \times n}$, where A_{ij} is the bond order between atoms i and j (0 = no bond, 1 = single, 2 = double, 3 = triple).

3.2 Chemical Axioms

The supervisor enforces four axioms. These are fundamental conservation laws and bonding rules from general chemistry:

Table 1: Chemical axioms enforced by the constraint checker.

Axiom	Formal rule	Meaning
Valency	$b_i + h_i \leq V(e_i, q_i)$	Every atom's total bond count (explicit bonds plus implicit hydrogens) must not exceed the maximum valency allowed for its element and charge. For example, neutral carbon allows 4; nitrogen with charge +1 allows 4.
Mass conservation	$\left\lvert \sum_{\text{react}} m - \sum_{\text{prod}} m \right\rvert \leq \epsilon$	The total atomic mass of reactants must equal the total atomic mass of products, within a small tolerance ϵ (default 0.02 u).
Charge conservation	$\sum_{\text{react}} q = \sum_{\text{prod}} q$	The net formal charge must be identical on both sides of a reaction.
Atom conservation	$\forall\, e:\ n_e^{\text{react}} = n_e^{\text{prod}}$	Every element must appear in the same count on both sides.

The effective valency function $V(e_i, q_i)$ accounts for charge-adjusted bonding capacity. Table 2 lists the ten elements currently supported along with their standard maximum valencies and how formal charge can alter them.

Table 2: Supported elements with atomic mass, maximum valency, and charge-adjusted valency where applicable.

Element	Mass (u)	Max valency	Charge adjustment
H	1.008	1	—
C	12.011	4	—
N	14.007	3	$+1 \to 4$; $-1 \to 2$
O	15.999	2	$-1 \to 1$
F	18.998	1	—
P	30.974	5	$+1 \to 6$
S	32.060	6	$+1 \to 7$; $+2 \to 8$
Cl	35.450	1	—
Br	79.904	1	—
I	126.904	1	—

3.3 The Generation Task

ChemCS operates at the level of chemical reactions, not individual molecules. Given a set of reactant molecules, the task is to generate a set of product molecules such that all four axioms in Table 1 are satisfied. The diffusion model operates on a single concatenated molecular graph (all reactant atoms merged into one graph), and the supervisor checks constraints at two levels:

- Per-atom (molecule-level): Valency is checked at every denoising step, since it depends only on the local bonding environment of each atom.

- Per-reaction: Mass, charge, and atom-count conservation are checked at the final step, since they require comparing the generated products against the original reactants.

This two-level design means the supervisor can catch local structural errors (overvalenced atoms) early in the trajectory, while global conservation errors are only detectable once the full product is available.

4 Methodology

4.1 Atom Feature Encoding

Each atom in a molecule is converted to a fixed-length feature vector of dimension 12. The first ten entries form a one-hot encoding of the element type; the remaining two entries encode the bond count and formal charge, each normalized to a standard range.

Table 3: Layout of the 12-dimensional atom feature vector.

Index	Feature	Encoding	Example
0–9	Element type	One-hot over 10 elements	Carbon $\rightarrow [0, 1, 0, \ldots, 0]$
10	Bond count	$b_i/4$	4 bonds $\rightarrow$ 1.0
11	Formal charge	$q_i/2$	$-1 \rightarrow -0.5$

The adjacency matrix A is constructed by greedily assigning bond orders between atom pairs based on each atom's remaining bonding capacity, producing a symmetric $n \times n$ matrix. The combination of $X \in \mathbb{R}^{n\times 12}$ (atom features) and $A \in \mathbb{R}^{n\times n}$ (bond orders) constitutes the complete molecular encoding.

4.2 GNN Denoiser Architecture

The denoiser is a two-layer graph neural network implemented in NumPy for lightweight CPU inference (a PyTorch version with identical architecture is used for training). Each graph convolutional layer updates every atom's hidden representation by combining two transformations:

$$h_i^{(\ell+1)} = \mathrm{ReLU}\left(W_{\text{self}}^{(\ell)} h_i^{(\ell)} + W_{\text{neigh}}^{(\ell)} \frac{1}{|\mathcal{N}(i)|} \sum_{j\in\mathcal{N}(i)} h_j^{(\ell)} \right) \tag{1}$$

where $h_i^{(\ell)}$ is the hidden vector of atom i at layer ℓ, $\mathcal{N}(i)$ is the set of neighbours of atom i (determined by the adjacency matrix), and $W_{\text{self}}^{(\ell)}$, $W_{\text{neigh}}^{(\ell)}$ are learnable weight matrices. The neighbour features are mean-aggregated (divided by the degree), which normalizes the contribution regardless of how many bonds an atom has.

Note that in Equation 1 the adjacency matrix A determines only the neighbourhood structure (which atoms are bonded), not the message weights. Bond orders are not used to scale the neighbour contributions. This is a simplification: in principle, a double bond should carry more information than a single bond. More expressive message-passing schemes that incorporate edge features (such as those used in DiGress) could improve the quality of the denoiser, but the current architecture is sufficient for the proof-of-concept evaluation.

After two graph convolutional layers, the network produces outputs through two heads:

- Atom head: A linear projection from the hidden dimension back to the 12-dimensional feature space, with softmax applied over the element logits (indices 0–9) to produce a probability distribution over element types.
- Bond head: For each pair of atoms (i, j), the hidden vectors h_i and h_j are concatenated into a single vector of size $2 \times$ hidden_dim, then passed through a linear layer with 4 outputs corresponding to bond orders 0–3. The predicted bond order is the argmax of these logits.

The default hidden dimension is 64, giving the model approximately 20,000 trainable parameters, which is small enough to run on CPU in under a second for molecules with up to roughly 15 atoms.

4.3 Noise Schedule

ChemCS supports linear and cosine noise schedules. The linear schedule increases β_t from 10^{-4} to 0.1; the cosine schedule follows $\bar{\alpha}_t = f(t)/f(0)$ with $f(u) = \cos^2\!\left(\frac{u/T+s}{1+s} \cdot \frac{\pi}{2}\right)$ and $s = 0.008$, giving a gentler ramp that improves sample quality. For atom features, noise is added via the standard DDPM rule $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$. For bond orders, each adjacency entry is re-sampled from $\{0, 1, 2, 3\}$ with probability $(1 - \bar{\alpha}_t)$, and symmetry is enforced by mirroring the upper triangle.

4.4 Constraint Checking with Z3

The constraint checker encodes each of the four chemical axioms (Table 1) as a Z3 satisfiability query. The checker works as follows:

1. Mass conservation: The mass difference between reactants and products is computed using Z3 real-valued arithmetic. The solver is asked whether the difference exceeds the tolerance ϵ. If the formula is satisfiable, mass is *not* conserved, and a violation is recorded.
2. Charge conservation: The total formal charges on both sides are compared as integers. A mismatch is recorded directly.
3. Atom conservation: For each element, the counts on the reactant and product sides are encoded as Z3 integer constants. The solver checks whether they differ; if satisfiable, a violation is recorded for that element.
4. Valency: For each atom in the products, the solver checks whether the total bond count (explicit bonds plus implicit hydrogens) exceeds the effective valency for that atom's element and formal charge. Per-atom violations are reported individually.

The function `check_intermediate` performs only the valency check (no reaction context needed) and is called at every denoising step. The full four-axiom check via `check_reaction` is invoked at the final step ($t = 1$) and after the trajectory completes.

Both functions return a `ConstraintResult` object containing a Boolean `sat` field and a list of human-readable violation strings, making the results easy to log and debug.

4.5 Supervisor Loop

The supervisor loop is the central contribution of ChemCS. It wraps the diffusion model's reverse process and intercepts every denoising step to enforce chemical validity. The procedure works as follows:

1. All reactant atoms are concatenated into a single initial state. This state is encoded into atom features X and adjacency matrix A, then noise is added up to timestep T.
2. From $t = T$ down to $t = 1$, the model performs one denoising step to produce a candidate state. The candidate is decoded back into a molecular representation and checked against

the relevant constraints (valency at every step; full mass, charge, and atom conservation at the final step).

3. If the candidate passes all checks, the step is committed and added to the trajectory history. If not, the supervisor attempts to repair the violations using the correction strategies described in Section 4.6.
4. If neither the original candidate nor any corrected version satisfies the constraints after the allowed number of retries, the supervisor backtracks: it reverts to the most recent valid state in the history and re-attempts from one timestep earlier, effectively undoing the failed step.
5. The process terminates when $t = 0$ is reached or when the maximum backtrack budget is exhausted.

The three possible outcomes for each timestep are:

- Commit: The denoised state passes all applicable checks on the first attempt. The step is accepted and appended to the trajectory history.
- Correct: The initial state fails a check, but one of the correction strategies (Section 4.6) repairs the violation. The corrected state replaces the original proposal.
- Backtrack: Neither the original proposal nor any corrected version passes. The supervisor reverts to the most recent valid state in the history and re-attempts from one timestep earlier.

The supervisor is parameterized by `max_retries` (how many correction attempts per timestep, default 3) and `max_backtracks` (global backtrack budget, default 10). These ensure the loop terminates even in adversarial cases.

4.6 Correction Strategies

When a denoising step produces an invalid state, the supervisor applies up to three targeted repairs before backtracking:

Table 4: Correction strategies applied by the supervisor.

Strategy	Targets	Mechanism
FixValency	Overvalenced atoms	Reduce explicit bonds (then implicit H) until the valency limit is met.
FixCharge	Charge imbalance	Distribute ± 1 charge adjustments across atoms (lightest first).
FixMass	Mass imbalance	Add/remove implicit hydrogens ($\approx$ 1.008 u each) to close the mass gap, respecting remaining bonding capacity.

These heuristics are intentionally simple and sufficient for the small molecules tested here; more sophisticated strategies are discussed in Section 8.

4.7 Constraint-Aware Training

When the diffusion model is trained only on a standard denoising objective, the supervisor must do all the work at inference time. To reduce the frequency of constraint violations, ChemCS augments the training loss with two differentiable penalty terms that encourage the model to produce chemically plausible outputs.

The total training loss is a weighted sum of four components:

$$\mathcal{L} = w_1 \cdot \mathcal{L}_{\text{denoise}} + w_2 \cdot \mathcal{L}_{\text{bond}} + w_3 \cdot \text{scale}(e) \cdot \mathcal{L}_{\text{val}} + w_4 \cdot \text{scale}(e) \cdot \mathcal{L}_{\text{elem}} \tag{2}$$

where scale(e) is the curriculum scaling factor (Equation 3) that ramps the constraint penalties from zero to their full weight over the course of training. Table 5 describes each term in detail.

Table 5: Components of the training loss function.

Term	Description	Default weight
$\mathcal{L}_{\text{denoise}}$	Mean squared error between the predicted atom features $\hat{x}_0$ and the clean target x_0. This is the standard denoising objective.	$w_1 = 1.0$
$\mathcal{L}_{\text{bond}}$	Cross-entropy loss between the predicted bond-order logits (4 classes: 0, 1, 2, 3) and the ground-truth adjacency matrix.	$w_2 = 1.0$
$\mathcal{L}_{\text{val}}$	Soft valency penalty: $\text{mean}\big(\text{ReLU}(\hat{b}_i - v_i)\big)$, where $\hat{b}_i$ is the expected total bond load (computed from softmax bond probabilities) and v_i is the expected allowed valency (computed from softmax element probabilities). Fully differentiable.	$w_3 = 0.5$
$\mathcal{L}_{\text{elem}}$	MSE between predicted and target element-count distributions, computed from the softmax of the element logits. Encourages the model to preserve the correct number of each element.	$w_4 = 0.3$

4.7.1 Curriculum Schedule

Introducing the constraint penalties ($\mathcal{L}_{\text{val}}$ and $\mathcal{L}_{\text{elem}}$) at full strength from the start of training destabilizes learning, because the model has not yet learned basic denoising and the penalty gradients dominate the loss landscape.

ChemCS uses a curriculum schedule that sets the constraint penalty weights to zero during a configurable warmup phase and then ramps them linearly to their full values over the remaining epochs:

$$\text{scale}(e) = \begin{cases} 0 & \text{if } e/E < f_{\text{warmup}} \\ \dfrac{e/E - f_{\text{warmup}}}{1 - f_{\text{warmup}}} & \text{otherwise} \end{cases} \tag{3}$$

where e is the current epoch, E is the total number of epochs, and f_{warmup} is the warmup fraction (default 25%). The effective penalty weights at epoch e are $w_3 \cdot \text{scale}(e)$ and $w_4 \cdot \text{scale}(e)$.

This allows the model to first learn how to denoise atom features and bond orders from the standard objectives, and only then begin incorporating chemical constraints once the representation is sufficiently stable.

4.7.2 Training Pipeline

The PyTorch training pipeline mirrors the NumPy model but adds a time embedding: the fractional timestep t/T is projected through a two-layer MLP with SiLU activation and added to the atom features. Training runs for a configurable number of epochs (default 100) with Adam (lr = 10^{-3}) and gradient clipping (max norm 1.0). After training, weights are exported to the NumPy model via `export_to_numpy()` for PyTorch-free inference.

5 Experimental Setup

5.1 Reaction Benchmark

Experiments use three representative chemical reactions that exercise different aspects of the constraint system:

Table 6: Benchmark reactions used for evaluation.

Reaction	Type	Atoms	Key challenge
$CH_3Br + OH^- \rightarrow ?$	SN2 nucleophilic substitution	7	Formal charge transfer (-1)
$CH_4 + 2\,O_2 \rightarrow ?$	Combustion	9	Multiple product molecules
$HCl + NaOH \rightarrow ?$	Acid-base neutralization	4	Mass balance with light atoms

5.2 Experimental Conditions

Each reaction is tested under two conditions at 50 random seeds:

- Unsupervised: Forward noise to T=10, then 10 reverse steps with no checks. The output is evaluated after completion.
- Supervised: Same model and noise, but wrapped in the supervisor loop with `max_retries=2` and `max_backtracks=3`.

Important: Both conditions instantiate a new GNN with random initialization (hidden dimension 32) for each seed. No checkpoint from the training run used for Figure 2 is loaded. The benchmark therefore measures supervisor vs. raw diffusion *before* any weight learning on the generation task.

5.3 Metrics

Three metrics are reported:

- Valency validity (%): Fraction of runs in which every atom in the product respects its effective valency limit. This is the primary metric and can be measured mid-trajectory.
- Full conservation (%): Fraction of runs in which all four axioms (mass, charge, atoms, valency) are satisfied. This is a much stricter criterion.
- Commit rate (%): Fraction of denoising steps that passed on the first attempt without correction or backtracking. This measures how readily each step was accepted by the supervisor.

5.4 Training Configuration (loss-curve experiment only)

To produce Figure 2, the PyTorch denoiser is trained on three small molecules: water (H_2O), methane (CH_4), and ethanol (C_2H_5OH). Settings: hidden dimension 32, 2 layers, 200 epochs, learning rate 10^{-3}, cosine noise schedule, 25% curriculum warmup. This run demonstrates that

the combined loss decreases and that the pipeline is implemented correctly. It is **not** the same experiment as Table 7: the table never uses the saved weights from this run.

6 Results

6.1 Constraint Checker Validation

The constraint checker is validated through a comprehensive test suite covering valid reactions (SN2, combustion), valency violations (e.g., carbon with 5 bonds), charge-adjusted valency (e.g., NH_4^+), conservation violations (missing products, extra atoms), and edge cases (empty products, high-valency elements). All tests pass, additionally covering encoding/decoding, noise schedules, reverse steps, correction heuristics, supervisor integration, the training loop, curriculum schedule, and model export.

6.2 Supervised vs. Unsupervised Generation

Figure 1 presents the main result: a comparison of valency validity between supervised and unsupervised generation across all three benchmark reactions.

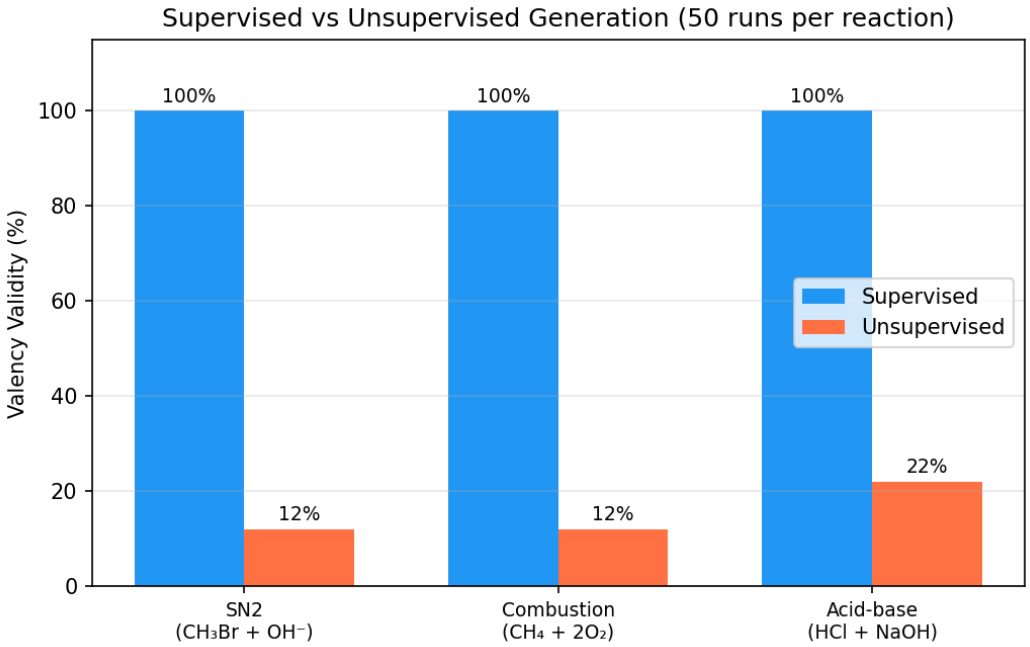

Figure 1: Valency validity (%) for supervised and unsupervised generation across three reactions, each tested with 50 random seeds using an untrained 32-dimensional GNN. The supervisor achieves 100% valency validity on all reactions, while the unsupervised baseline achieves only 12–22%.

Table 7 provides the complete numeric results:

Table 7: Benchmark results: 50 random seeds per reaction, untrained GNN with hidden dimension 32, T=10 diffusion steps.

	Valency validity		Full conservation	
Reaction	Supervised	Unsupervised	Supervised	Unsupervised
SN2	100%	12%	0%	0%
Combustion	100%	12%	0%	0%
Acid-base	100%	22%	0%	0%
Average	100%	15%	0%	0%

It is important to be precise about what these results do and do not show.

The 100% valency validity result means that the supervisor successfully enforces a local, per-atom structural constraint: no atom in any generated product has more bonds than its element allows. This is a necessary condition for chemical validity, but not a sufficient one. The supervisor achieves this by inspecting and repairing each atom independently, which it can do regardless of whether the GNN produces meaningful outputs. In effect, the experiment demonstrates that the supervisor can enforce local bonding rules on arbitrary molecular graphs, even those generated by random noise.

The 0% full conservation result is **not** because training was skipped in general—it is because this benchmark deliberately uses **untrained** weights only (Section 5). Mass, charge, and atom-count conservation are global constraints: they need the correct multiset of elements in the product. With random weights the network's element logits are meaningless, so decoding picks wrong elements and global checks fail even when valency is repaired locally.

The training run behind Figure 2 already shows that the loss (including denoising, bond, and penalty terms) can decrease on a tiny dataset. That does **not** by itself change Table 7: we did not plug those trained weights into the same 50-seed benchmark. The natural follow-up—load `export_to_numpy()` weights into the supervisor loop and repeat Table 7—is still outstanding and would test whether full conservation moves above 0%.

6.3 Training Dynamics

Figure 2 shows the training loss over 200 epochs on H_2O, CH_4, and C_2H_5OH *only*—again, this curve is not coupled to the random-weight benchmark above.

During the warmup phase (epochs 1–50), only the denoising MSE and bond cross-entropy are active; the MSE converges quickly. When the valency penalty is activated at epoch 50 it initially spikes, then decreases as the model adapts. By epoch 100 all components stabilize, with bond cross-entropy dominating due to the $O(n^2)$ pairwise prediction.

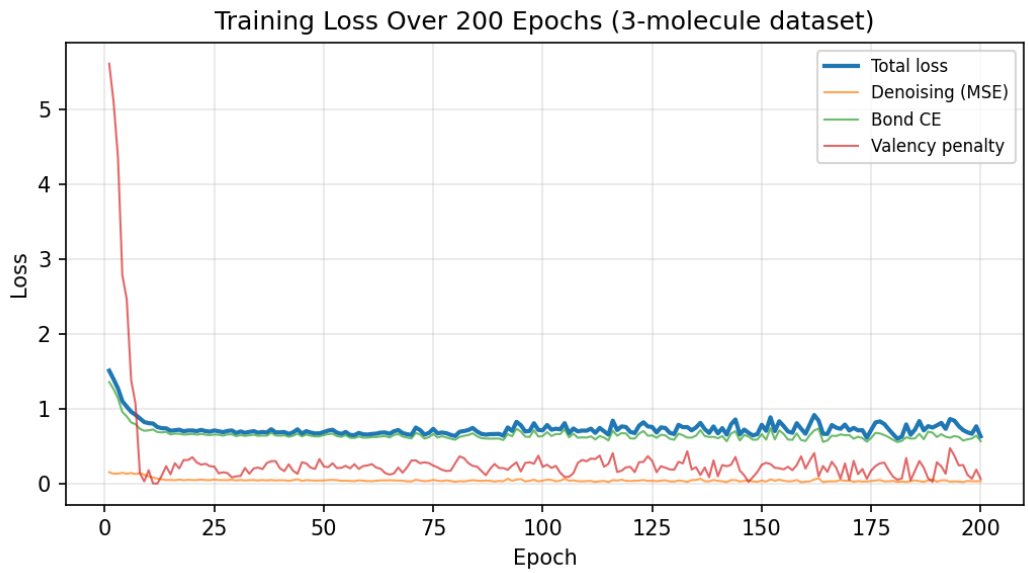

Figure 2: Training loss components over 200 epochs on water, methane, and ethanol.

Figure 3 compares three curriculum warmup fractions. The 25% warmup was selected as the default: no warmup caused loss spikes (constraint penalties overwhelmed denoising gradients), while 50% warmup left too few epochs for the penalties to take effect.

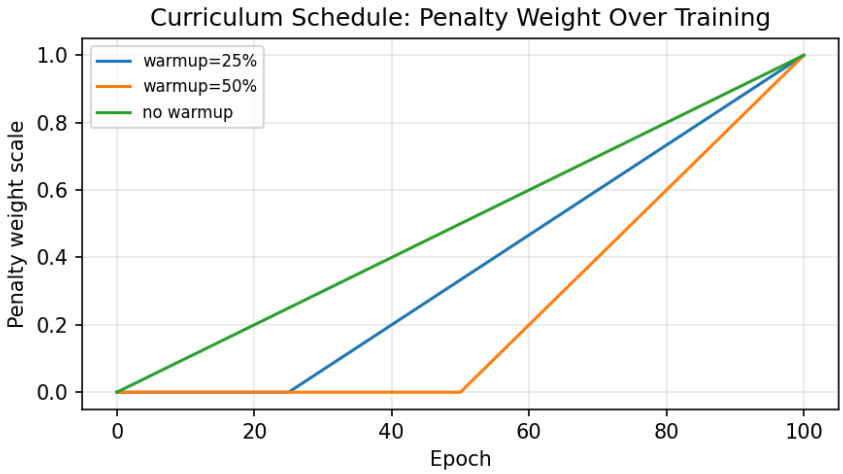

Figure 3: Curriculum penalty scale for three warmup strategies (0%, 25%, 50%) over 100 epochs.

7 Discussion

The central finding is that constraint enforcement during generation enforces local structural validity even with an untrained network. The 100% valency result demonstrates that the commit/correct/backtrack loop works correctly as an engineering system; the 0% full conservation result shows its limits, since global constraints require a trained model that predicts correct element types.

The results support a broader design principle: logic and learning are complementary. The neural network provides generative capacity, while the constraint system provides domain knowledge. Neither alone suffices—a constraint checker cannot propose novel molecules, and a generator without constraints produces mostly invalid outputs. We expect that exporting trained weights into the same benchmark script could raise full-conservation rates once the network learns sensible element logits; that end-to-end evaluation is not part of the numbers reported here.

The constraint-aware training loss bridges inference-time and training: it gives the model gradient signal about chemical validity so that over time the supervisor has fewer violations to correct. The curriculum schedule is essential because the model must learn basic denoising before it can meaningfully respond to valency penalties.

8 Limitations and Future Work

Data and model scale. Training uses only three molecules; a serious evaluation would require a standard dataset such as QM9 or ZINC. The two-layer, 20k-parameter GNN with mean-aggregation is sufficient for the proof of concept but unlikely to generalise. Scaling to edge-aware message passing (as in DiGress) and comparing against established baselines (DiGress, GDSS, MolGAN) are necessary.

Correction sophistication. The repair heuristics are greedy and local. A more principled approach would formulate correction itself as a constraint-satisfaction problem, asking Z3 for a minimal edit set.

Benchmark breadth and molecular complexity. The 50-seed, 3-reaction benchmark is too small for strong statistical conclusions, and the system handles only molecules up to $\sim$15 atoms with no rules for ring structures, stereochemistry, or aromaticity.

End-to-end trained evaluation. The highest-priority extension is to run the *same* supervised vs. unsupervised protocol while loading weights from `export_to_numpy()` after training on a richer set of graphs, so full conservation can be compared fairly against the 0% untrained baseline in Table 7.

9 Conclusion

This paper presented ChemCS, a proof-of-concept system that integrates constraint satisfaction directly into the reverse-diffusion loop of a graph neural network molecular generator. By checking valency limits at every denoising step, the supervisor enforces 100% local bonding validity on a small benchmark of three reactions, compared with 12–22% for the same untrained model without supervision. Global conservation constraints (mass, charge, atom counts) remain unsatisfied with random weights, highlighting that the supervisor addresses structural validity but not chemical correctness, which requires a trained model.

A companion training pipeline adds differentiable constraint penalties through a curriculum schedule. We ran it to obtain Figure 2, which shows the loss decreasing on three small molecules—evidence that the trainer works. Table 7 and Figure 1 remain **untrained-only**; connecting trained weights to that benchmark is left for future work.

The architecture demonstrates that combining symbolic reasoning (via Z3) with neural generation (via a GNN denoiser) can enforce local domain constraints without sacrificing the flexibility of the generative model. Whether this approach scales to larger models, larger molecules, and more complex constraint sets is an open question that this work is designed to motivate. The code, figures, and tests are available in the project repository.

References

[1] E. Hoogeboom, V. G. Satorras, C. Vignac, and M. Welling, "Equivariant diffusion for molecule generation in 3D," in *Proc. International Conference on Machine Learning (ICML)*, vol. 162, pp. 8867–8887, 2022.
`https://arxiv.org/abs/2203.17003`

[2] C. Vignac, I. Krawczuk, A. Siraudin, B. Wang, V. Cevher, and P. Frossard, "DiGress: Discrete denoising diffusion for graph generation," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
`https://arxiv.org/abs/2209.14734`

[3] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proc. International Conference on Machine Learning (ICML)*, vol. 139, pp. 8162–8171, 2021.
`https://arxiv.org/abs/2102.09672`

[4] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture Notes in Computer Science, vol. 4963, pp. 337–340, 2008.
`https://link.springer.com/chapter/10.1007/978-3-540-78800-3_24`

[5] R. Ramakrishnan, P. O. Dral, M. Rupp, and O. A. von Lilienfeld, "Quantum chemistry structures and properties of 134 kilo molecules," *Scientific Data*, vol. 1, no. 140022, 2014.
`https://www.nature.com/articles/sdata201422`

[6] J. J. Irwin, T. Sterling, M. M. Mysinger, E. S. Bolstad, and R. G. Coleman, "ZINC: A free tool to discover chemistry for biology," *Journal of Chemical Information and Modeling*, vol. 52, no. 7, pp. 1757–1768, 2012.
`https://pubs.acs.org/doi/10.1021/ci3001277`